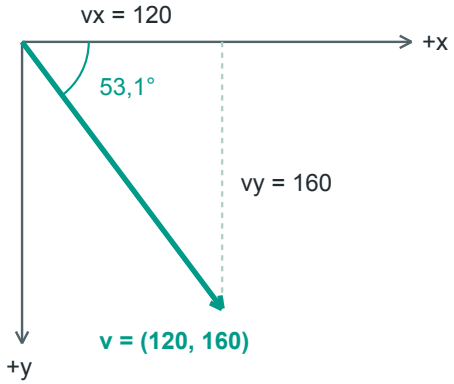
Matematika pohybu hráče

Poloha, vektor, úhel, rychlost a zrychlení v souřadnicích obrazovky.

1 Co uchováváme a v jakých jednotkách

p = (x, y) je poloha [px], **v = (vx, vy)** rychlost [px/s], **a = (ax, ay)** zrychlení [px/s²]. Vektor má dvě složky; sčítání a násobení číslem provádíme pro každou zvlášť.



Velikost rychlosti (délka vektoru):

```
s = math.sqrt(vx*vx + vy*vy)
```

Pro v = (120, 160) vyjde **s = 200 px/s**. Jednotkový směr **u = v / s = (0,6; 0,8)** má délku 1. Nulový vektor nemá určený směr a nelze jej normalizovat.

Na obrazovce roste **x doprava a y dolů**. Úhel zde měříme od kladné osy x po směru hodinových ručiček.

2 Směr z kláves nebo z úhlu

Pro šipky jsou dx a dy hodnoty -1, 0 nebo 1. Diagonála (1, 1) má délku √2: bez normalizace je pohyb asi **1,41krát rychlejší**.

```
d = pygame.Vector2(dx, dy)
if d.length() > 0:
    d = d.normalize()
v = d * 200
```

Rychlost má velikost 200 px/s při pohybu; pro d = (0, 0) je nulová. Směr z úhlu **ve stupních** vypočteme takto:

```
r = math.radians(uhel)
d = pygame.Vector2(math.cos(r),
                   math.sin(r))
```

0° vpravo, 90° dolů, 180° vlevo. Opačný převod pro nenulový směr:

```
uhel = math.degrees(math.atan2(dy, dx))
```

Obrázek `image0` mířící původně doprava natočíme pomocí `rotate(image0, -uhel)`. Funkce pygame.transform.rotate otáčí kladně proti směru hodin.

3 Posun = rychlost × uplynulý čas

```
dt = clock.tick(60) / 1000
p += v * dt
```

`tick()` voláme **jednou za snímek**; vrátí ms, dělením 1000 dostaneme sekundy. Při stálé v rozhoduje uplynulý čas, nikoli počet snímků.

Příklad: p = (100, 100), v = (120, 160), dt = 0,25 s. Posun je (30, 40) px, nová poloha **(130, 140)**.

Desetiny zachováme v p typu Vector2. Rect ukládá celé souřadnice. Čtyři kroky po 0,25 px dají 1 px; malé kroky proto nepřičítáme přímo k Rectu.

4 Zrychlení mění rychlost

Zrychlení mění rychlost. Jednoduchý výpočet po časových krocích:

```
v += a * dt
p += v * dt
```

Z klidu: a = (300, 0), dt = 0,1 s. Nová v je (30, 0) px/s, tento krok posune p o (3, 0) px. Jde o aproximaci; kratší kroky obvykle zmenšují chybu.

Při stálém a je přesně $\Delta p = v_0 \cdot dt + \frac{1}{2} a \cdot dt^2$, zde (1,5; 0) px; v_0 je rychlost před krokem.

5 Krátký výřez pro sprite

Výřez bez událostí a kreslení; pygame, okno a image již existují. p znamená **střed obrázku**.

```
# jednou pred smyckou
p = pygame.Vector2(100, 100)
v = pygame.Vector2(0, 0)
a = pygame.Vector2(300, 0)
rect = image.get_rect(center=p)
clock = pygame.time.Clock()
# v kazdem snimku
dt = clock.tick(60) / 1000
v += a * dt
p += v * dt
rect.center = (round(p.x), round(p.y))
```

6 Tlumení a limit rychlosti

Tlumení bez zrychlení: `v *= 0.5 ** dt` ponechá za 1 sekundu polovinu rychlosti i při jiném FPS.

```
if v.length() > limit:
    v.scale_to_length(limit)
```

limit > 0 je v px/s. Oba kroky patří mezi změnu rychlosti a posun polohy.

Souvislosti: 04_hrac_v_plose používá krok za snímek; 08_Image počítá rychlost × dt. 06_CAR ukazuje vektor a úhel, ale má dt = tick()/10 a poloha += rychlost. Zde je jiný model: dt v sekundách.

API pygame-ce: vektory čas rotace Rect